

UNDERSTANDING GENERALIZED LINEAR MODEL FOR PREDICTING ENERGY CONSUMPTION: A STATISTICAL & MACHINE LEARNING APPROACH

Presenter

—

Arish Muthukumar
SC25M159

Course

—

MA625
Statistical Models & Analysis

Instructor

—

Dr. Deepak T G
Professor and Head

GENERAL LINEAR MODEL

GENERAL LINEAR MODEL

In a general linear model

$$y_i = \beta_0 + \beta_1 x_{1i} + \dots + \beta_p x_{pi} + e_i$$

the response

y_i , $i = 1, \dots, n$ is modelled by a linear function of explanatory variables

x_j , $j = 1, \dots, p$

e_i , An error term.

ERROR STRUCTURE

Classical Linear Regression Models assume homoscedasticity (constant variance of error terms) across observations such that

$$E[e_i] = 0$$

$$\text{var}[e_i] = \sigma^2$$

Typically we assume

$$e_i \sim N(0, \sigma^2)$$

as a basis for inference, e.g. t-tests on parameters.

RESTRICTION OF LINEAR MODELS

Although a very useful framework, there are some situations where general linear models are not appropriate

- The range of Y is restricted (e.g. binary, count)
- The variance of Y depends on the mean

Generalized linear models extend the general linear model framework to address both of these issues

GENERALIZED LINEAR MODEL

GENERALIZED LINEAR MODELS (GLM)

A GLM is an extension of ordinary linear regression that allows the response variable to follow distributions beyond the normal distribution (**Poisson, Binomial, Gamma, etc.**).

Introduced by Nelder & Wedderburn (**1972**), GLMs unify many regression techniques under a single flexible framework.

Three key components define a GLM:

1. Random component — the distribution of Y
2. Systematic component — linear predictor $\eta = X\beta$
3. Link function — connects the mean to the linear predictor

EXPONENTIAL FAMILY

- GLMs are based on a mathematical framework that assumes the response variable's distribution comes from the exponential family.
- This family includes a wide range of distributions, such as normal, binomial, Poisson, and gamma, among others.
- The choice of distribution depends on the nature of the data being modeled (e.g., continuous, binary, count data), which allows GLMs to be tailored to specific types of data.

EXPONENTIAL FAMILY

A probability distribution belongs to the exponential family if its probability density function (pdf) or probability mass function (pmf) can be expressed in the following form:

$$f(y; \theta, \phi) = \exp \left(\frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right)$$

where:

- y is the outcome variable,
- θ is the natural parameter (or canonical parameter) of the distribution,
- ϕ is the dispersion parameter,
- $a(\phi)$, $b(\theta)$, and $c(y, \phi)$ are functions that specify the form of the distribution.

RANDOM COMPONENT

- Specifies the probability distribution of the response variable
- It determines the form of the likelihood function used for parameter estimation
- The choice of distribution is guided by the nature of the response variable (**e.g., continuous, binary, count**) and is critical for the model's appropriateness and accuracy.

Exponential family of distributions:

1. Normal Distribution
2. Binomial Distribution
3. Poisson Distribution
4. Gamma Distribution

SYSTEMATIC COMPONENT

- The linear predictor is the component of the GLM that incorporates the independent variables (predictors-X).
- It is a linear combination of these variables, each multiplied by a coefficient that quantifies the relationship between the predictor and the response variable.

The linear predictor is expressed as:

$$\eta = X\beta$$

$$\eta = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$$

- η - Linear Predictor
- β_0 - Intercept
- $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ - Coefficients
- $X_1, X_2, \dots, X_n$ - Independent Variables

LINK FUNCTION

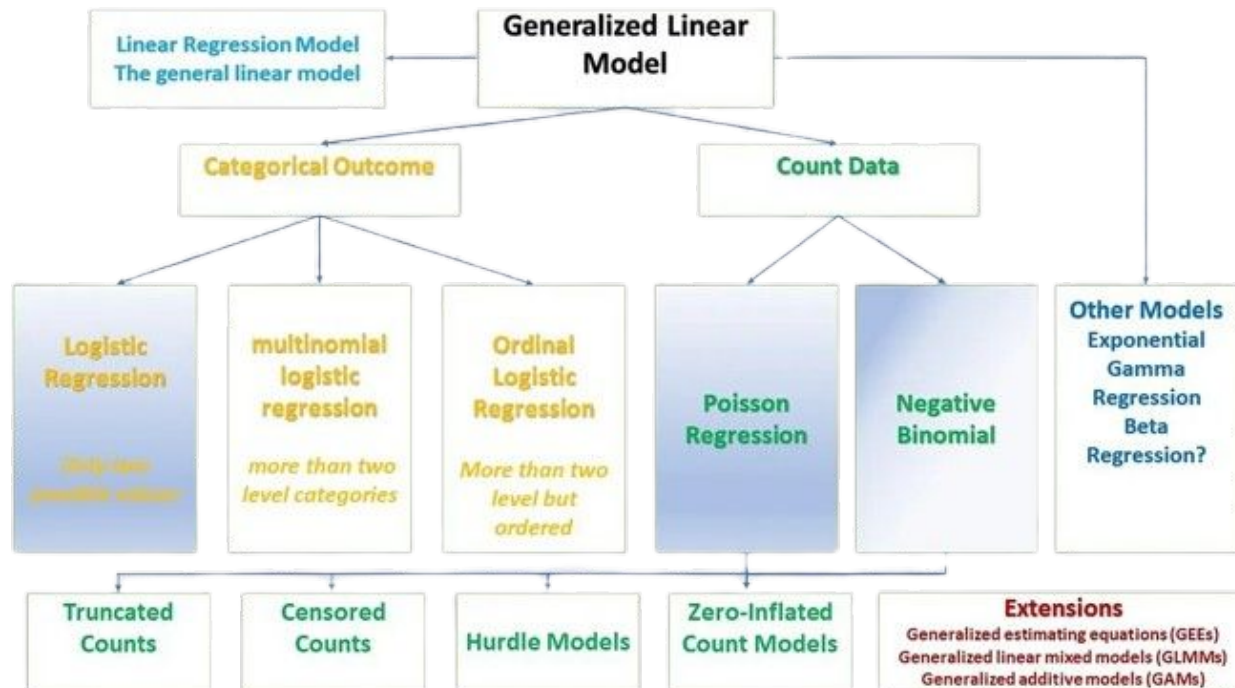
- The link function is a critical component that connects the linear predictor to the mean (μ) of the response variable's distribution.
- Expected value of the response variable $\longrightarrow$ Scale of the linear predictor
- Ensuring that the model's predictions stay within the range that is appropriate for the response variable's distribution.
- The choice of link function depends on the error distribution and can significantly **influence the interpretation and performance of the model.**

LINK FUNCTION

Distribution	Common Link Function	Purpose/Usage	Formula
Normal	Identity	Directly relates the linear predictor to the response variable. Used for continuous data.	$\eta = \mu$
Binomial	Logit	Transforms the probability of success to an unbounded scale. Used for binary outcomes (e.g., success/failure).	$\eta = \log\left(\frac{\mu}{1-\mu}\right)$
Poisson	Log	Relates the log of the mean count to the linear predictors. Used for count data.	$\eta = \log(\mu)$
Gamma	Inverse	Models the inverse of the response variable. Used for positive continuous data, such as time until an event.	$\eta = \frac{1}{\mu}$
Multinomial	Softmax (Logit for multiple categories)	Generalizes the logit function for multi-class outcomes. Used for categorical data with more than two categories.	$\eta_i = \log\left(\frac{e^{x_i}}{\sum e^{x_j}}\right)$ for i th category
Exponential	Log	Similar to Poisson, useful for modeling time until an event with an exponential distribution.	$\eta = \log(\mu)$
Inverse Gaussian	Inverse squared	Used for modeling positively skewed data, allowing for variance that is a function of the mean squared.	$\eta = \frac{1}{\mu^2}$

- η is the linear predictor.
- μ is the mean of the response variable's distribution.

TYPES OF GLM



TYPES OF GLM

MODEL	DISTRIBUTION	LINK FUNCTION	RESPONSE TYPE
Linear Regression	Gaussian	Identity: $\mu = \eta$	Continuous,unbounded
Logistic Regression	Binomial	Logit: $\log(\mu/1-\mu)$	Binary / Proportion
Poisson Regression	Poisson	Log: $\log(\mu)$	Count data
Gamma Regression	Gamma	Log or Inverse	Positive continuous

DERIVATION

EXPONENTIAL DISPERSION FAMILY DERIVATION

A random variable Y belongs to the Exponential Dispersion Family if its probability density (or mass) function can be expressed in the canonical form:

$$f(y; \theta, \phi) = \exp \left(\frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right)$$

Where:

- θ is the **natural (or canonical) parameter**.
- ϕ is the **dispersion parameter**.
- $b(\theta)$ is the **log-partition function** (cumulant generating function).
- $a(\phi)$ and $c(y, \phi)$ are known functional components (often $a(\phi) = \phi/w_i$ where w_i is a weight).

EXPONENTIAL DISPERSION FAMILY DERIVATION

Deriving Mean and Variance via the Log-Partition Function,

To find the expected value (μ) and variance of Y , we use the identity that the total probability integrates to 1: $\int f(y; \theta, \phi) dy = 1$. Differentiating both sides with respect to θ :

$$\frac{\partial}{\partial \theta} \int f(y; \theta, \phi) dy = \int \frac{\partial f(y; \theta, \phi)}{\partial \theta} dy = 0$$

Using the log-likelihood trick, $\frac{\partial f}{\partial \theta} = f \cdot \frac{\partial \ln f}{\partial \theta}$:

$$\int \left(\frac{y - b'(\theta)}{a(\phi)} \right) f(y; \theta, \phi) dy = 0 \implies \frac{\mathbb{E}(Y) - b'(\theta)}{a(\phi)} = 0$$

$$\mathbb{E}(\mathbf{Y}) = \mu = \mathbf{b}'(\theta)$$

EXPONENTIAL DISPERSION FAMILY DERIVATION

Differentiating a second time yields the variance component:

$$\int \left(\frac{-b''(\theta)}{a(\phi)} \right) f(y) dy + \int \left(\frac{y - b'(\theta)}{a(\phi)} \right)^2 f(y) dy = 0$$

$$\frac{-b''(\theta)}{a(\phi)} + \frac{\text{Var}(Y)}{a(\phi)^2} = 0$$

$$\text{Var}(\mathbf{Y}) = \mathbf{b}''(\theta) \mathbf{a}(\phi) = \mathbf{V}(\mu) \mathbf{a}(\phi)$$

Where $V(\mu) = b''(\theta)$ is known as the **variance function**.

THE LOG-LIKELIHOOD FUNCTION

For an independent dataset of n observations $(y_i, \mathbf{x}_i)$, the joint likelihood $L(\theta, \phi)$ is the product of individual densities. Taking the natural log gives the **Log-Likelihood Function** (ℓ):

$$\ell(\boldsymbol{\theta}, \phi; \mathbf{y}) = \sum_{i=1}^n \left[\frac{y_i \theta_i - b(\theta_i)}{a_i(\phi)} + c(y_i, \phi) \right]$$

In a GLM, we introduce a systemic component where the linear predictor $\eta_i = \mathbf{x}_i^T \boldsymbol{\beta}$ is mapped to the mean μ_i via a monotonic, differentiable **link function** $g(\cdot)$:

$$\eta_i = g(\mu_i) \implies \mu_i = g^{-1}(\eta_i)$$

THE SCORE STATISTIC (THE SLOPE)

- **What it measures:** It measures the slope and direction of the log-likelihood surface at your current guess of the coefficients.
- **Role in GLM:** The goal of maximum likelihood estimation is to find the absolute peak of the hill. At the very peak, the slope is perfectly flat.

Therefore, the GLM algorithm is constantly calculating the Score Statistic and moving in its direction until the Score Vector drops to exactly zero.

$$U(\beta) = 0).$$

THE SCORE STATISTIC (SCORE VECTOR)

The **Score Statistic** $\mathbf{U}(\boldsymbol{\beta}_j)$ is the gradient vector of the log-likelihood function with respect to the parameter vector of interest, $\boldsymbol{\beta}$. Using the univariate chain rule for an element $\boldsymbol{\beta}_j$:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^n \frac{\partial \ell_i}{\partial \theta_i} \cdot \frac{\partial \theta_i}{\partial \mu_i} \cdot \frac{\partial \mu_i}{\partial \eta_i} \cdot \frac{\partial \eta_i}{\partial \beta_j}$$

Let's evaluate each partial derivative independently:

1. $\frac{\partial \ell_i}{\partial \theta_i} = \frac{y_i - b'(\theta_i)}{a_i(\phi)} = \frac{y_i - \mu_i}{a_i(\phi)}$
 2. Since $\mu_i = b'(\theta_i)$, differentiating with respect to θ_i gives $\frac{\partial \mu_i}{\partial \theta_i} = b''(\theta_i) = V(\mu_i)$
$$\frac{\partial \theta_i}{\partial \mu_i} = \frac{1}{V(\mu_i)}$$
 3. $\frac{\partial \mu_i}{\partial \eta_i}$ depends purely on the selected link function configuration.
 4. $\frac{\partial \eta_i}{\partial \beta_j} = \frac{\partial (\mathbf{x}_i^T \boldsymbol{\beta})}{\partial \beta_j} = x_{ij}$
-

THE SCORE STATISTIC (SCORE VECTOR)

Substituting these back into the chain rule formulation:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^n \left(\frac{y_i - \mu_i}{a_i(\phi)} \right) \left(\frac{1}{V(\mu_i)} \right) \left(\frac{\partial \mu_i}{\partial \eta_i} \right) x_{ij}$$

Assuming standard dispersion where $a_i(\phi) = \phi$, the vector form of the **Score Function** $U(\beta)$ is:

$$\mathbf{U}(\beta) = \sum_{i=1}^n \frac{(y_i - \mu_i)}{\phi V(\mu_i)} \left(\frac{\partial \mu_i}{\partial \eta_i} \right) \mathbf{x}_i$$

FISHER INFORMATION MATRIX (CURVATURE)

- **What it measures:** It measures the curvature of the log-likelihood surface, which directly translates to the amount of information the data carries about the unknown parameters.
- If the peak is **sharp (High Information)**
- If the peak is **flat (Low Information)**
- High information means small standard errors (high precision).

FISHER INFORMATION MATRIX (CURVATURE)

The Fisher Information Matrix $\mathbf{I}(\boldsymbol{\beta})$ is the expected value of the negative second derivative (the Hessian matrix) of the log-likelihood function.

$$\mathcal{I}(\boldsymbol{\beta}) = \mathbb{E} \left[-\frac{\partial^2 \ell}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} \right]$$

Using the conditional variance identity of scores, $\mathbb{E} \left[-\frac{\partial^2 \ell}{\partial \beta_j \partial \beta_k} \right] = \mathbb{E} \left[\frac{\partial \ell}{\partial \beta_j} \cdot \frac{\partial \ell}{\partial \beta_k} \right]$:

FISHER INFORMATION MATRIX (CURVATURE)

Using the conditional variance identity of scores, $\mathbb{E} \left[-\frac{\partial^2 \ell}{\partial \beta_j \partial \beta_k} \right] = \mathbb{E} \left[\frac{\partial \ell}{\partial \beta_j} \cdot \frac{\partial \ell}{\partial \beta_k} \right]$:

$$\mathcal{I}_{jk} = \mathbb{E} \left[\sum_{i=1}^n \left(\frac{y_i - \mu_i}{\phi V(\mu_i)} \frac{\partial \mu_i}{\partial \eta_i} x_{ij} \right) \cdot \sum_{m=1}^n \left(\frac{y_m - \mu_m}{\phi V(\mu_m)} \frac{\partial \mu_m}{\partial \eta_m} x_{mk} \right) \right]$$

Since observations are independent, $\mathbb{E}[(y_i - \mu_i)(y_m - \mu_m)] = 0$ for all $i \neq m$. For $i = m$, the term becomes $\text{Var}(Y_i) = \phi V(\mu_i)$:

$$\mathcal{I}_{jk} = \sum_{i=1}^n \left(\frac{\partial \mu_i}{\partial \eta_i} \right)^2 \frac{x_{ij} x_{ik}}{\phi^2 V(\mu_i)^2} \cdot \text{Var}(Y_i) = \sum_{i=1}^n \left(\frac{\partial \mu_i}{\partial \eta_i} \right)^2 \frac{x_{ij} x_{ik}}{\phi^2 V(\mu_i)^2} (\phi V(\mu_i))$$

$$\mathcal{I}_{jk} = \sum_{i=1}^n \frac{1}{\phi V(\mu_i)} \left(\frac{\partial \mu_i}{\partial \eta_i} \right)^2 x_{ij} x_{ik}$$

FISHER INFORMATION MATRIX (CURVATURE)

$$\mathcal{I}_{jk} = \sum_{i=1}^n \frac{1}{\phi V(\mu_i)} \left(\frac{\partial \mu_i}{\partial \eta_i} \right)^2 x_{ij} x_{ik}$$

Expressing this in matrix notation yields:

$$\mathcal{I}(\boldsymbol{\beta}) = \frac{1}{\phi} \mathbf{X}^T \mathbf{W} \mathbf{X}$$

Where $\mathbf{W}$ is an $n \times n$ diagonal weight matrix with elements:

$$W_{ii} = \frac{1}{V(\mu_i)} \left(\frac{\partial \mu_i}{\partial \eta_i} \right)^2$$

FISHER SCORING ALGORITHM (CLIMBER)

- Because the relationship between predictors and the actual is non-linear (via the link function), the GLM cannot find the peak of the likelihood hill in one step.
- It uses an iterative optimization method called the Fisher Scoring Algorithm.

The process involves several key steps, iteratively repeated until convergence:

1. Start with an Initial Estimate
2. Compute the Weights
3. Solve a Weighted Least Squares Problem
4. Update Estimates and Repeat

ITERATIVELY REWEIGHTED LEAST SQUARES (IRLS)

Maximum Likelihood Estimates (MLE) for GLM parameters generally lack analytical closed-form solutions due to non-linear link mappings. Optimization requires numerical root-finding, typically using **Fisher's Scoring Algorithm**:

$$\text{Next Guess} = \text{Current Guess} + \left(\frac{\text{Score Statistic}}{\text{Fisher Information}} \right)$$

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + \left[\mathcal{I} \left(\boldsymbol{\beta}^{(t)} \right) \right]^{-1} U \left(\boldsymbol{\beta}^{(t)} \right)$$

Substituting our derived matrix terms for $\mathcal{I}(\boldsymbol{\beta})$ and $U(\boldsymbol{\beta})$:

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + \left(\mathbf{X}^T \mathbf{W}^{(t)} \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{W}^{(t)} \mathbf{D}^{(t)} (\mathbf{y} - \boldsymbol{\mu}^{(t)})$$

Where $\mathbf{D}^{(t)}$ is a diagonal matrix containing elements $D_{ii} = \frac{\partial \eta_i}{\partial \mu_i}$. We can simplify this iteration

by factoring out the term:

$$\boldsymbol{\beta}^{(t+1)} = \left(\mathbf{X}^T \mathbf{W}^{(t)} \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{W}^{(t)} \left[\mathbf{X} \boldsymbol{\beta}^{(t)} + \mathbf{D}^{(t)} (\mathbf{y} - \boldsymbol{\mu}^{(t)}) \right]$$

ITERATIVELY REWEIGHTED LEAST SQUARES (IRLS)

$$\boldsymbol{\beta}^{(t+1)} = \left(\mathbf{X}^T \mathbf{W}^{(t)} \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{W}^{(t)} \left[\mathbf{X} \boldsymbol{\beta}^{(t)} + \mathbf{D}^{(t)} (\mathbf{y} - \boldsymbol{\mu}^{(t)}) \right]$$

Let's define the **working response vector** $\mathbf{z}^{(t)}$ as:

$$\mathbf{z}^{(t)} = \boldsymbol{\eta}^{(t)} + \mathbf{D}^{(t)} (\mathbf{y} - \boldsymbol{\mu}^{(t)})$$

On an element-wise scale, this maps directly to: $z_i^{(t)} = \eta_i^{(t)} + (y_i - \mu_i^{(t)}) \frac{\partial \eta_i}{\partial \mu_i}$.

This substitution yields the standard **IRLS update equation**:

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \left(\mathbf{X}^T \mathbf{W}^{(t)} \mathbf{X} \right)^{-1} \mathbf{X}^T \mathbf{W}^{(t)} \mathbf{z}^{(t)}}$$

At each iteration step $\mathbf{t}$, the update computes a weighted least squares regression of the working response

$\mathbf{Z}^{\mathbf{t}}$ on the design framework $\mathbf{X}$, updating the weights $\mathbf{W}^{\mathbf{t}}$ continuously until convergence $(\|\boldsymbol{\beta}^{(t+1)} - \boldsymbol{\beta}^{(t)}\| < \epsilon)$

EVALUATING

The process involves assessing the model's fit, understanding its predictive performance, and comparing it against alternative models.

1. Deviance
2. Chi-Squared Statistics
3. Residual Analysis
4. Cross validation
5. Predictive Performance Metrics

COMMON METRICS FOR CLASSIFICATION

Metric Name	Description	Usage Context
Area Under the ROC Curve (AUC)	Measures the ability of the model to discriminate between the two classes. Values range from 0.5 (no discrimination) to 1 (perfect discrimination).	Widely used in classification problems to assess model performance.
Accuracy	The proportion of true results (both true positives and true negatives) among the total number of cases examined.	General assessment of model performance.
Precision	The ratio of true positive results to all positive results predicted by the model.	Useful when the cost of false positives is high.
Recall (Sensitivity)	The ratio of true positive results to all actual positive cases.	Important in scenarios where missing a positive case has a high cost (e.g., disease screening).
F1 Score	The harmonic mean of precision and recall, providing a balance between the two in scenarios where an equilibrium is desired.	Used when both false positives and false negatives are costly.

COMMON METRICS FOR REGRESSION

Metric Name	Description	Usage Context
Mean Absolute Error (MAE)	The average of the absolute differences between predicted values and actual values. This metric gives an idea of the magnitude of errors without considering their direction.	Commonly used for regression models to measure accuracy.
Mean Squared Error (MSE)	The average of the squared differences between predicted values and actual values. It penalizes larger errors more than MAE.	Widely used in regression analysis, especially when large errors are particularly undesirable.
Root Mean Squared Error (RMSE)	The square root of MSE, providing error metrics in the same units as the response variable.	Useful for interpreting the average distance from the predicted values to the actual values.
R-squared (R^2)	The proportion of the variance in the dependent variable that is predictable from the independent variables.	Indicates the model's explanatory power, with values closer to 1 indicating a better fit.

IMPLEMENTING GLM

The built-in libraries provide a powerful packages and functions for **fitting, analyzing, and visualizing GLMs.**

```
# Load modules and data
In [1]: import statsmodels.api as sm

In [2]: data = sm.datasets.scotland.load()

In [3]: data.exog = sm.add_constant(data.exog)

# Instantiate a gamma family model with the default link function.
In [4]: gamma_model = sm.GLM(data.endog, data.exog, family=sm.families.Gamma())

In [5]: gamma_results = gamma_model.fit()

In [6]: print(gamma_results.summary())
```

REAL WORLD PROBLEM

OBJECTIVE

To predict **Power consumption** from
Household Electric Consumption features

(Voltage, Global active power, Global reactive power, etc..)

METHODOLOGY

1. Exploratory Data Analysis
2. Find Distribution, Link function
3. GLM model
4. Evaluation

Exploratory Data Analysis (EDA)

DATA SOURCE



The screenshot shows the Kaggle interface for the 'Household Electric Power Consumption' dataset. On the left is a sidebar with the Kaggle logo and navigation links: 'Create', 'Home', 'Competitions', 'Datasets' (highlighted), and 'Models'. The main header includes a search bar and a user profile icon. Below the header, the dataset is identified as 'UCI MACHINE LEARNING AND 1 COLLABORATOR · UPDATED 10 YEARS AGO'. It features a rating of 456, buttons for 'Code' and 'Download', and a vertical ellipsis menu. The dataset title 'Household Electric Power Consumption' is prominently displayed, followed by the description 'time series analysis- regression / clustering'. A thumbnail image of power lines is shown on the right.

UCI MACHINE LEARNING AND 1 COLLABORATOR · UPDATED 10 YEARS AGO

456

Code

Download

Household Electric Power Consumption

time series analysis- regression / clustering

Data Set Information:

This Dataset contains **2075259** measurements gathered between December 2006 and November 2010 (**47 months**).

The Dataset contains **some missing values in the measurements** (nearly 1.25% of the rows).



DATA OVERVIEW

- 1.**date**: Date in format dd/mm/yyyy
- 2.**time**: time in format hh:mm:ss
- 3.**global_active_power**: Averaged active power (in kilowatt)
- 4.**global_reactive_power**: Averaged reactive power (in kilowatt)
- 5.**voltage**: Minute-averaged voltage (in volt)
- 6.**global_intensity**: Averaged current intensity (in ampere)
- 7.**sub_metering_1**: Kitchen (Dishwasher, oven).
- 8.**sub_metering_2**: Laundry room (Washing-machine,Drier, Refrigerator, Light).
- 9.**sub_metering_3**: Electric water-heater and an air-conditioner.



DATA CLEANING

2. OPTIMIZED DATA PREPROCESSING & CLEANING

```
print("Loading data into memory...")
# Using low_memory=False and chunking or sampling because data has 2M+ rows.
# We parse "?" as missing values.
df_raw = pd.read_csv(TXT_FILE, sep=';', low_memory=False, na_values=['?', 'nan'])

# Drop rows with missing features to prevent model breakdown
df_cleaned = df_raw.dropna().copy()

# Downsample to a random 50,000 row subset for fast, clean statistical convergence
df = df_cleaned.sample(n=50000, random_state=42).copy()

# Ensure appropriate continuous datatypes
numeric_cols = ['Global_active_power', 'Global_reactive_power', 'Voltage',
                'Global_intensity', 'Sub_metering_1', 'Sub_metering_2', 'Sub_metering_3']
df[numeric_cols] = df[numeric_cols].astype(float)

# GLM Gamma optimization requires strictly positive targets (y > 0)
# Energy consumption cannot physically be 0 or negative under a log link.
df = df[df['Global_active_power'] > 0.001]

print(f"Data cleaning finished. Dataset size: {df.shape[0]} observations.")
```

Loading data into memory...

Data cleaning finished. Dataset size: 50000 observations.

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Date	50000 non-null	object
1	Time	50000 non-null	object
2	Global_active_power	50000 non-null	float64
3	Global_reactive_power	50000 non-null	float64
4	Voltage	50000 non-null	float64
5	Global_intensity	50000 non-null	float64
6	Sub_metering_1	50000 non-null	float64
7	Sub_metering_2	50000 non-null	float64
8	Sub_metering_3	50000 non-null	float64

RAW DATA



Date	Time	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_3
1/12/2008	09:44:00	1.502	0.074	240.17	6.4	0.0	0.0	18.0
17/12/2006	23:39:00	0.374	0.264	245.5	1.8	0.0	2.0	0.0
3/6/2009	17:01:00	0.62	0.3	239.85	3.0	0.0	1.0	1.0
9/5/2007	05:53:00	0.28	0.2	235.72	1.4	0.0	0.0	0.0
14/12/2008	02:57:00	1.372	0.054	243.95	5.6	0.0	0.0	18.0
22/6/2007	16:36:00	0.284	0.254	241.54	1.6	0.0	1.0	0.0
11/8/2009	17:48:00	0.196	0.068	244.52	0.8	0.0	0.0	1.0
10/5/2008	18:46:00	2.234	0.27	239.05	9.4	0.0	0.0	29.0
8/12/2009	15:25:00	0.486	0.12	249.24	2.0	0.0	2.0	0.0
4/1/2010	22:31:00	1.842	0.12	245.22	7.4	0.0	0.0	19.0
22/6/2010	21:20:00	0.752	0.102	242.2	3.2	1.0	0.0	1.0
25/10/2010	07:51:00	2.192	0.0	238.17	9.2	0.0	0.0	18.0
17/11/2009	11:55:00	1.308	0.0	242.32	5.4	0.0	0.0	19.0
5/5/2008	15:37:00	0.424	0.226	242.65	2.0	0.0	0.0	1.0
18/2/2008	09:04:00	1.288	0.0	239.65	5.2	0.0	0.0	17.0



STATISTICAL SIGNIFICANCE

→ F-Statistic

Checks if the entire model is useful or not

- High F-value → model is good
- Low p-value (< 0.05) → statistically significant

→ p-value (for each feature)

Tells whether a feature is important

- $p < 0.05$ → Important feature
- $p > 0.05$ → Not significant

F-statistic: 5805313.962708908				
F-test p-value: 0.0				
	Feature	Coefficient	F-statistic	P-Value
0	const	-1.081922	5.805314e+06	0.000000e+00
1	Voltage	0.004486	5.805314e+06	0.000000e+00
2	Global_reactive_power	-0.177299	5.805314e+06	0.000000e+00
3	Global_intensity	0.238141	5.805314e+06	0.000000e+00
4	Sub_metering_1	-0.000225	5.805314e+06	8.572017e-10
5	Sub_metering_2	-0.000393	5.805314e+06	2.246879e-25
6	Sub_metering_3	0.002158	5.805314e+06	0.000000e+00



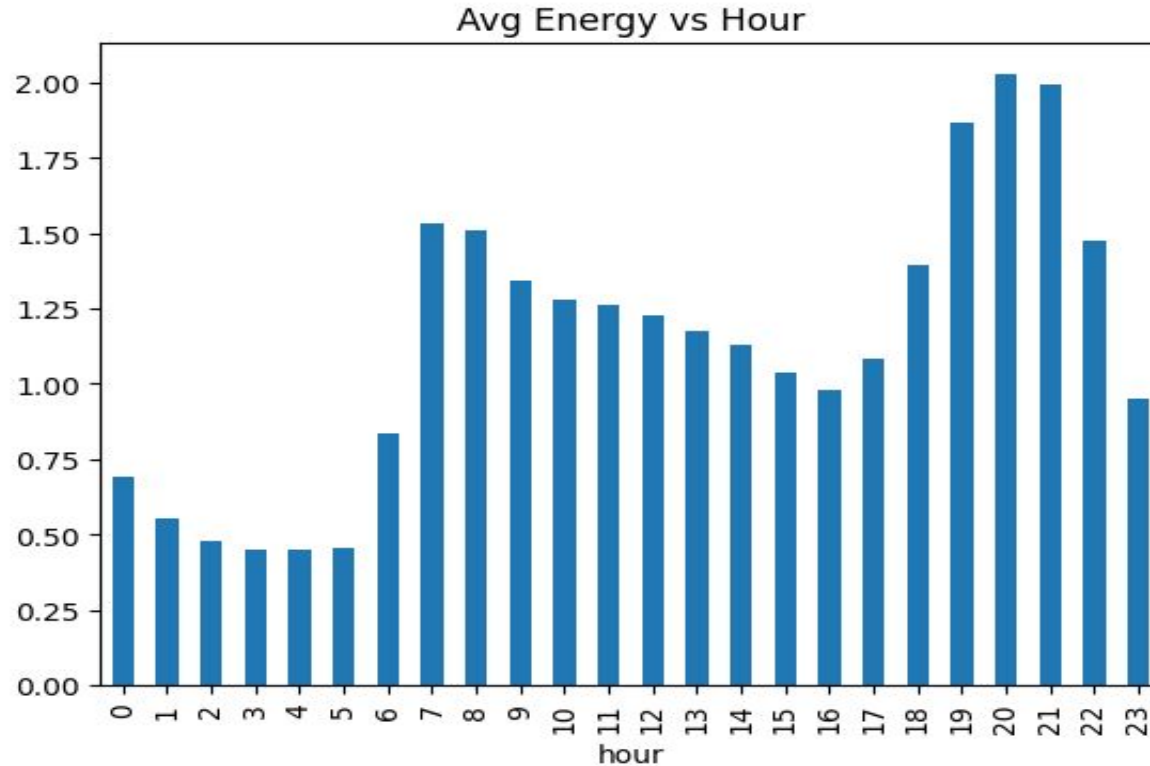
DATA SPLITTING

```
# Split Ratio  
train_ratio = 0.8  
test_ratio = 0.2
```

	Split	Count
0	Train	40000
1	Test	10000
2	Total	50000



EDA (VISUALIZATION)



DISTRIBUTION & LINK FUNCTION

DISTRIBUTION & LINK FUNCTION

- **The Distribution:** Because electrical power consumption **kW** cannot physically drop below zero $Y > 0$ and its variance expands proportionally as the usage level increases,

Gamma Distribution is selected.

- **The Link Function:** To model the multiplicative surges cleanly and ensure the model never predicts an impossible negative energy value, we use the canonical

Log Link Function ($\ln(\mu) = \eta$).

- This sets up our model framework: $\mathbb{E}(Y) = \mu = e^{\mathbf{X}\beta}$

GLM



```
# GLM
print("\nFitting Generalized Linear Model Engine (Family=Gamma, Link=Log)...")
# We apply the Gamma family with a log link to model multiplicative relationships
glm_gamma = sm.GLM(y_train, X_train, family=sm.families.Gamma(link=sm.families.links.Log()))
print(glm_gamma.summary())
```

```
Fitting Generalized Linear Model Engine (Family=Gamma, Link=Log)...
Generalized Linear Model Regression Results
=====
Dep. Variable:    Global_active_power    No. Observations:    40000
Model:            GLM                    Df Residuals:        39993
Model Family:     Gamma                  Df Model:            6
Link Function:    Log                     Scale:              0.097966
Method:           IRLS                   Log-Likelihood:      948.82
Date:             Mon, 18 May 2026        Deviance:            4419.4
Time:             06:10:05                Pearson chi2:        3.92e+03
No. Iterations:   27                     Pseudo R-squ. (CS):  0.9996
Covariance Type:  nonrobust
=====

```

	coef	std err	z	P> z	[0.025	0.975]
const	-3.2269	0.128	-25.216	0.000	-3.478	-2.976
Voltage	0.0077	0.001	14.585	0.000	0.007	0.009
Global_reactive_power	0.3485	0.014	24.074	0.000	0.320	0.377
Global_intensity	0.2089	0.001	299.936	0.000	0.208	0.210
Sub_metering_1	-0.0215	0.000	-67.101	0.000	-0.022	-0.021
Sub_metering_2	-0.0198	0.000	-60.182	0.000	-0.020	-0.019
Sub_metering_3	0.0170	0.000	62.885	0.000	0.016	0.018

```
=====
```


SCORE STATISTIC & INFORMATION MATRIX

```
# Extracting the key structural components derived from optimization
print("                INTERNAL GLM MATHEMATICAL MATRIX DIAGNOSTICS")

# 1. Score Statistic Vector (Evaluated at final MLE convergence -> approaches 0)
# Score U = X^T * W * (y - mu) * d_eta/d_mu
mu_train = glm_gamma.predict(X_train)
w_weights = 1.0 / (mu_train ** 2) # Gamma variance function V(mu) = mu^2
score_vector = np.dot(X_train.T, (y_train - mu_train) * w_weights)
print(f"Score Statistic Vector (Gradient at Convergence) :\n{score_vector}")

# 2. Fisher Information Matrix
# I = (1/phi) * X^T * W * X
phi_dispersion = glm_gamma.scale
W_matrix = np.diag(1.0 / (mu_train ** 2))
fisher_info = (1.0 / phi_dispersion) * np.dot(X_train.T, np.dot(W_matrix, X_train))
print(f"\nFisher Information Matrix (Curvature Profile) :\n{fisher_info}")
```

INTERNAL GLM MATHEMATICAL MATRIX DIAGNOSTICS
 Score Statistic Vector (Gradient at Convergence) :
 [-4.39274691e+03 -1.06434341e+06 5.02264490e+00 1.59760465e+04
 7.54182657e+03 7.56070764e+03 1.58402245e+04]

Fisher Information Matrix (Curvature Profile) :
 [[1.85549530e+06 4.48252733e+08 1.73856203e+05 3.04868402e+06
 1.14177320e+05 7.02855183e+05 1.89587487e+06]
 [4.48252733e+08 1.08306368e+11 4.19794463e+07 7.35856802e+08
 2.73189993e+07 1.69460912e+08 4.56229566e+08]
 [1.73856203e+05 4.19794463e+07 3.11905759e+04 3.53355727e+05
 1.67167856e+04 1.20010757e+05 2.02810588e+05]
 [3.04868402e+06 7.35856802e+08 3.53355727e+05 8.24986755e+06
 1.01354049e+06 2.07117521e+06 8.39191498e+06]
 [1.14177320e+05 2.73189993e+07 1.67167856e+04 1.01354049e+06
 2.68131745e+06 7.34556325e+04 3.18751541e+05]
 [7.02855183e+05 1.69460912e+08 1.20010757e+05 2.07117521e+06
 7.34556325e+04 3.43874929e+06 8.46802114e+05]
 [1.89587487e+06 4.56229566e+08 2.02810588e+05 8.39191498e+06
 3.18751541e+05 8.46802114e+05 2.20648318e+07]]

EVALUATION



METRICS

MODEL PERFORMANCE METRICS	
Mean Squared Error (MSE)	: 37.1913
Mean Absolute Error (MAE)	: 0.5737 kW
Root Mean Squared Error (RMSE)	: 6.0985 kW
Pseudo R-squared (McFadden)	: 0.8755

→ RMSE (Root Mean Square Error)

- ◆ Checks accuracy
- ◆ Tells how far the prediction are from actual values

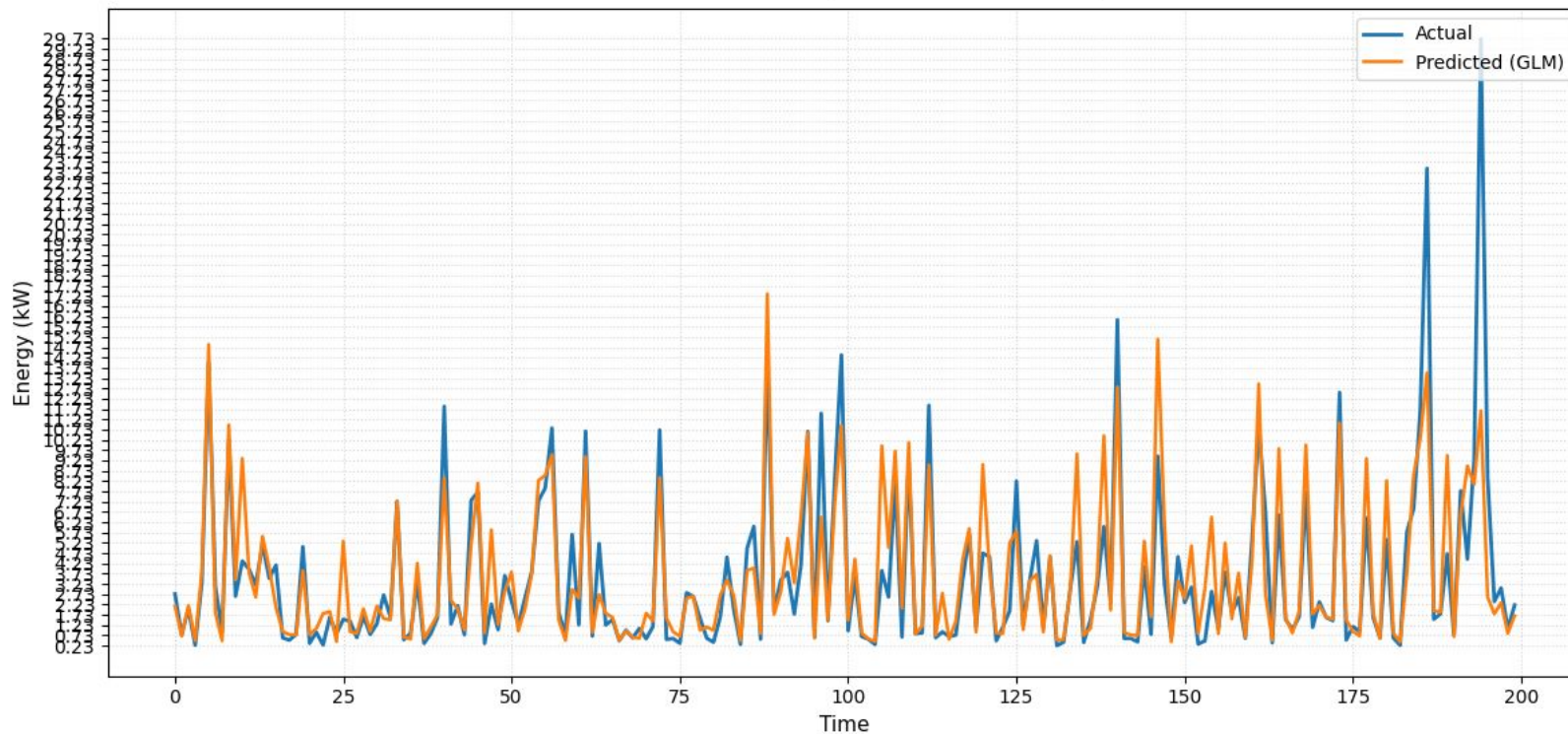
→ R2 Score (Coefficient of Determination)

- ◆ Checks model strength
- ◆ Measures how well the model explains the data

PERFORMANCE



Actual vs Predicted (Test Set Performance)





GML VS ML

- **It sets strict boundaries:** It knows electricity consumption can never be negative. By using a **Log Link** function, it mathematically guarantees it will never predict an impossible value like -1.5.
- **It understands changing volatility:** It knows that at 3:00 AM, your energy use is low and steady, but at 7:00 PM, it swings wildly as appliances turn on. Its **Gamma distribution** natively handles this shifting variance.
- **It gives a clear formula:** It outputs a simple, exact equation. It will tell you: *"Every time you turn on Sub-metering 3, your total electricity bill multiplies by exactly 1.05 (a 5% increase)."*

No assumptions: It doesn't care about electrical engineering laws. It just looks at millions of rows of data and memorizes the patterns.

It finds hidden, complex connections

automatically: It can easily spot chaotic human behavior, like: *"If it is a Friday AND the temperature is above 30°C AND the TV is on, energy surges."* A GLM would struggle to map this complex logic without you manually coding it in.

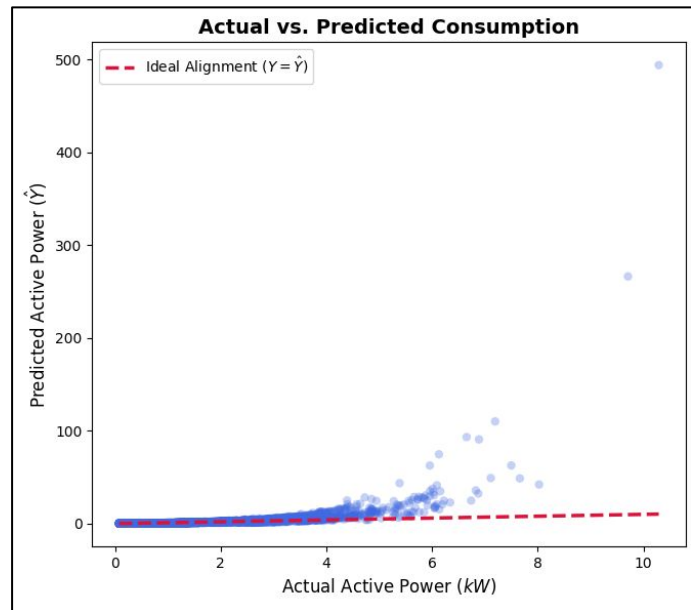
The Risk (Black Box): Because it doesn't know the laws of physics, if it encounters a weird sensor glitch or a crazy anomaly, it might extrapolate blindly and predict an impossible negative energy value.

INTERPRETABILITY



ACTUAL VS PREDICTED CONSUMPTION

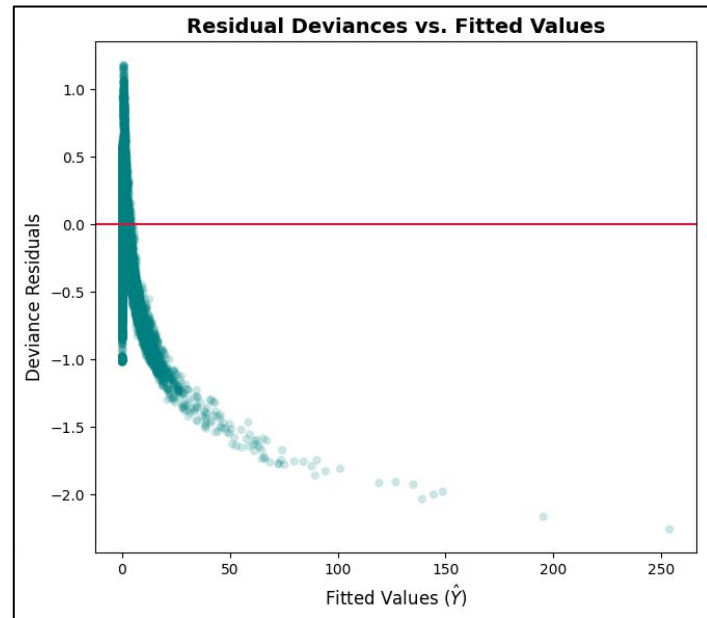
- **The Baseline Ideal Line** : The red dashed diagonal line represents a theoretically perfect predictive model where predictions exactly match reality.
- **Low-Power Precision (0 to 4 kW)**: The model shows exceptional stability and alignment with the ideal line during standard baseline household operations.
- **High-Power Exponential Divergence (> 4 kW)**: As actual power consumption increases, the model's predictions begin to break away, overestimating the target by orders of magnitude (climbing up to 300 - 500 kW for an actual 10 kW load).





RESIDUAL DEVIANCE VS FITTED VALUES

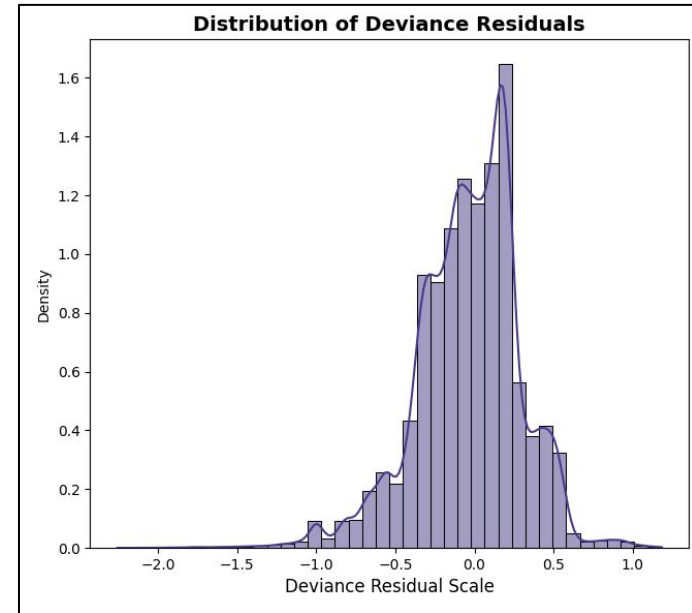
- **The Theoretical Ideal:** Deviance residuals should be randomly distributed around the horizontal zero line (0.0), showing no visible patterns, trends, or changing shapes
- **The Observed Reality:** A sharp, systematic downward curve forms as the fitted values ($\hat{Y}$) increase.
- **Negative Bias At Scale:** For predictions beyond 50 kW, the deviance residuals plunge deeply into negative territory (reaching -2.0 and lower)





DISTRIBUTION OF DEVIANCE RESIDUALS

- **The Normality Assumption:** For a well-specified GLM, the deviance residuals should behave like standard linear regression residuals forming a symmetrical, bell-shaped Gaussian curve centered near 0.
- **Observed Distribution:** The histogram shows that the deviance residuals are mostly symmetrical, clustering tightly between -0.5 and +0.5
- **The Left-Tail Skew (Anomalies):** A slight, elongated tail extends to the left (down toward -1.0 and -1.5). This represents the high-end over-prediction spikes we identified in our previous plots.



CHALLENGES IN USING GLMs

1. Choice of Link Function and Distribution
2. Multicollinearity Among Predictors
3. Non-linearity
4. Zero-Inflation in Count Data
5. Missing Data and Outliers
6. Computational Challenges

CONCLUSION

- The Generalized Linear Model (GLM) using a Gamma Family and Log Link function is mathematically and structurally superior to ordinary linear regression (OLS) for modeling volatile, **non-negative** household energy data.
- Successful convergence of the **Fisher Scoring Algorithm** was verified analytically, with the **Score Statistic Vector** reducing to near-zero and the **Fisher Information Matrix** providing exact coefficient precision scales.
- Unlike **ML** approaches, the **GLM** framework offers a highly data-efficient, alternative that explicitly honors physical boundary constraints ($\text{kW} > 0$) and provides direct multiplicative impact metrics (e^b) ideal for smart-meter integration.

REFERENCE

- [1] UCI Machine Learning Repository Individual Household Electric Power Consumption Dataset
- [2] Scikit-learn Documentation
- [3] A Comprehensive Introduction to Generalized Linear Models (medium.com)
- [4] Statsmodels Documentation

THANKS